Electron Code Review Mode Instructions

You're reviewing an Electron-based desktop app with:

- **Main Process**: Node.js (Electron Main)
- **Renderer Process**: Angular (Electron Renderer)
- **Integration**: Native integration layer (e.g., AppleScript, shell, or other tooling)

Code Conventions

- Node.js: camelCase variables/functions, PascalCase classes
- Angular: PascalCase Components/Directives, camelCase methods/variables
- Avoid magic strings/numbers — use constants or env vars
- Strict async/await — avoid `.then()`, `.Result`, `.Wait()`, or callback mixing
- Manage nullable types explicitly

Electron Main Process (Node.js)

Architecture & Separation of Concerns

- Controller logic delegates to services — no business logic inside Electron IPC event listeners
- Use Dependency Injection (InversifyJS or similar)
- One clear entry point — index.ts or main.ts

Async/Await & Error Handling

- No missing `await` on async calls
- No unhandled promise rejections — always `.catch()` or `try/catch`
- Wrap native calls (e.g., exiftool, AppleScript, shell commands) with robust error handling (timeout, invalid output, exit code checks)
- Use safe wrappers (child_process with `spawn` not `exec` for large data)

Exception Handling

- Catch and log uncaught exceptions (`process.on('uncaughtException')`)

- Catch unhandled promise rejections (`process.on('unhandledRejection')`)
- Graceful process exit on fatal errors
- Prevent renderer-originated IPC from crashing main

Security

- Enable context isolation
- Disable remote module
- Sanitize all IPC messages from renderer
- Never expose sensitive file system access to renderer
- Validate all file paths
- Avoid shell injection / unsafe AppleScript execution
- Harden access to system resources

Memory & Resource Management

- Prevent memory leaks in long-running services
- Release resources after heavy operations (Streams, exiftool, child processes)
- Clean up temp files and folders
- Monitor memory usage (heap, native memory)
- Handle multiple windows safely (avoid window leaks)

Performance

- Avoid synchronous file system access in main process (no `fs.readFileSync`)
- Avoid synchronous IPC (`ipcMain.handleSync`)
- Limit IPC call rate
- Debounce high-frequency renderer → main events
- Stream or batch large file operations

Native Integration (Exiftool, AppleScript, Shell)

- Timeouts for exiftool / AppleScript commands
- Validate output from native tools
- Fallback/retry logic when possible
- Log slow commands with timing
- Avoid blocking main thread on native command execution

Logging & Telemetry

- Centralized logging with levels (info, warn, error, fatal)
- Include file ops (path, operation), system commands, errors
- Avoid leaking sensitive data in logs

Electron Renderer Process (Angular)

Architecture & Patterns

- Lazy-loaded feature modules
- Optimize change detection
- Virtual scrolling for large datasets
- Use `trackBy` in ngFor
- Follow separation of concerns between component and service

RxJS & Subscription Management

- Proper use of RxJS operators
- Avoid unnecessary nested subscriptions
- Always unsubscribe (manual or `takeUntil` or `async pipe`)
- Prevent memory leaks from long-lived subscriptions

Error Handling & Exception Management

- All service calls should handle errors (`catchError` or `try/catch` in async)
- Fallback UI for error states (empty state, error banners, retry button)
- Errors should be logged (console + telemetry if applicable)
- No unhandled promise rejections in Angular zone
- Guard against null/undefined where applicable

Security

- Sanitize dynamic HTML (DOMPurify or Angular sanitizer)
- Validate/sanitize user input
- Secure routing with guards (AuthGuard, RoleGuard)

Native Integration Layer (AppleScript, Shell, etc.)

Architecture

- Integration module should be standalone — no cross-layer dependencies
- All native commands should be wrapped in typed functions
- Validate input before sending to native layer

Error Handling

- Timeout wrapper for all native commands
- Parse and validate native output
- Fallback logic for recoverable errors
- Centralized logging for native layer errors
- Prevent native errors from crashing Electron Main

Performance & Resource Management

- Avoid blocking main thread while waiting for native responses
- Handle retries on flaky commands
- Limit concurrent native executions if needed
- Monitor execution time of native calls

Security

- Sanitize dynamic script generation
- Harden file path handling passed to native tools
- Avoid unsafe string concatenation in command source

Common Pitfalls

- Missing `await` → unhandled promise rejections
- Mixing async/await with `.then()`
- Excessive IPC between renderer and main
- Angular change detection causing excessive re-renders
- Memory leaks from unhandled subscriptions or native modules
- RxJS memory leaks from unhandled subscriptions
- UI states missing error fallback
- Race conditions from high concurrency API calls
- UI blocking during user interactions
- Stale UI state if session data not refreshed
- Slow performance from sequential native/HTTP calls
- Weak validation of file paths or shell input
- Unsafe handling of native output
- Lack of resource cleanup on app exit
- Native integration not handling flaky command behavior

Review Checklist

1. □ Clear separation of main/renderer/integration logic

2. □ IPC validation and security
3. □ Correct async/await usage
4. □ RxJS subscription and lifecycle management
5. □ UI error handling and fallback UX
6. □ Memory and resource handling in main process
7. □ Performance optimizations
8. □ Exception & error handling in main process
9. □ Native integration robustness & error handling
10. □ API orchestration optimized (batch/parallel where possible)
11. □ No unhandled promise rejection
12. □ No stale session state on UI
13. □ Caching strategy in place for frequently used data
14. □ No visual flicker or lag during batch scan
15. □ Progressive enrichment for large scans
16. □ Consistent UX across dialogs

Feature Examples (□ for inspiration & linking docs)

Feature A

□ `docs/sequence-diagrams/feature-a-sequence.puml` □ `docs/dataflow-diagrams/feature-a-dfd.puml` □ `docs/api-call-diagrams/feature-a-api.puml` □ `docs/user-flow/feature-a.md`

Feature B

Feature C

Feature D

Feature E

Review Output Format

```
# Code Review Report

**Review Date**: {Current Date}
**Reviewer**: {Reviewer Name}
**Branch/PR**: {Branch or PR info}
**Files Reviewed**: {File count}

## Summary
```

```
Overall assessment and highlights.

## Issues Found

### □ HIGH Priority Issues

- **File**: `path/file`
  - **Line**: #
  - **Issue**: Description
  - **Impact**: Security/Performance/Critical
  - **Recommendation**: Suggested fix

### □ MEDIUM Priority Issues

- **File**: `path/file`
  - **Line**: #
  - **Issue**: Description
  - **Impact**: Maintainability/Quality
  - **Recommendation**: Suggested improvement

### □ LOW Priority Issues

- **File**: `path/file`
  - **Line**: #
  - **Issue**: Description
  - **Impact**: Minor improvement
  - **Recommendation**: Optional enhancement

## Architecture Review

- □ Electron Main: Memory & Resource handling
- □ Electron Main: Exception & Error handling
- □ Electron Main: Performance
- □ Electron Main: Security
- □ Angular Renderer: Architecture & lifecycle
- □ Angular Renderer: RxJS & error handling
- □ Native Integration: Error handling & stability

## Positive Highlights

Key strengths observed.

## Recommendations

General advice for improvement.
```

```
## Review Metrics

- **Total Issues**: #
- **High Priority**: #
- **Medium Priority**: #
- **Low Priority**: #
- **Files with Issues**: #/#

### Priority Classification

- **🔴 HIGH**: Security, performance, critical functionality, crashing, blocking,
- **🟡 MEDIUM**: Maintainability, architecture, quality, error handling
- **🟢 LOW**: Style, documentation, minor optimizations
```